CSCI 3060U Phase 4

Back End Rapid Prototype

Design Document

Group Members:
Mansi Kandoi 100821953
Sneh Patel 100950609
Jolly Soni 100876863

Table of Contents

Table of Contents *1*
1. Purpose *2*
2. Inputs and Outputs *2*
Inputs 2
Output 2
3. Architecture Overview *3*
4. UML Class Diagram *4*
5. Class and Method Descriptions *5*
6. How to Run *7*

1. Purpose

The Back End is a Python command-line program that completes the banking system pipeline begun by the Front End. It reads the Old Master Bank Accounts File (BAF) and the merged Accumulated Transaction File (ATF) that the Front End sessions produced, applies every transaction in order, and writes an updated New Master Bank Accounts File for the next business day.

2. Inputs and Outputs

Inputs

Old Master Bank Accounts File: one record per line, fixed-width 45-character format:

```
NNNNN AAAAAAAAAAAAAAAAAAAA S BBBBBBBB TTTT PP
```

Field	Name	Description
NNNNN	Account Number	5-digit zero-padded account number
A...A	Account Holder	20-character name, left-justified, space-padded
S	Status	A = active, D = disabled
BBBBBBBB	Balance	Balance in XXXXX.XX format (8 characters)
TTTT	Transaction Count	4-digit cumulative transaction count
PP	Plan	SP = student plan, NP = non-plan

The last line is always the END_OF_FILE sentinel record (account number 00000).

Merged Accumulated Transaction File (ATF), one record per line, fixed-width 40-character format:

```
CC AAAAAAAAAAAAAAAAAAAA NNNNN BBBBBBBBMM
```

Field	Name	Description
CC	Transaction Code	2-digit code: 0108 (transaction), 00 (end-of-session)
A...A	Account Holder	20-character account holder name
NNNNN	Account Number	5-digit account number (00000 for create)
BBBBBBBB	Amount	Transaction amount in XXXXX.XX format
MM	Misc	Bill company (EC/CQ/FI), transfer destination suffix, or spaces

Output

New Master Bank Accounts File same 45-character fixed-width format as the input BAF, with account balances and transaction counts updated to reflect all applied transactions. The file always ends with the END_OF_FILE sentinel record.

3. Architecture Overview

The Back End is organized as six Python source files, each with a single clearly defined responsibility. main.py is the entry point: it parses the three command-line arguments and delegates the entire pipeline to BackendApp. BackendApp (backend_app.py) acts as the controller, sequentially calling the reader, processor, and writer. AccountFileReader (read.py) parses the BAF into Account objects. TransactionFileReader (read_transactions.py) parses the ATF into Transaction objects. TransactionProcessor (process.py) applies every transaction to the in-memory account collection and returns the updated list. AccountFileWriter (write.py) serialises the final Account list back to the fixed-width BAF format. ErrorLogger (print_error.py) is a shared utility called by all modules to print consistently formatted error messages; fatal errors cause an immediate exit.

Data flows through the pipeline as plain Python objects:

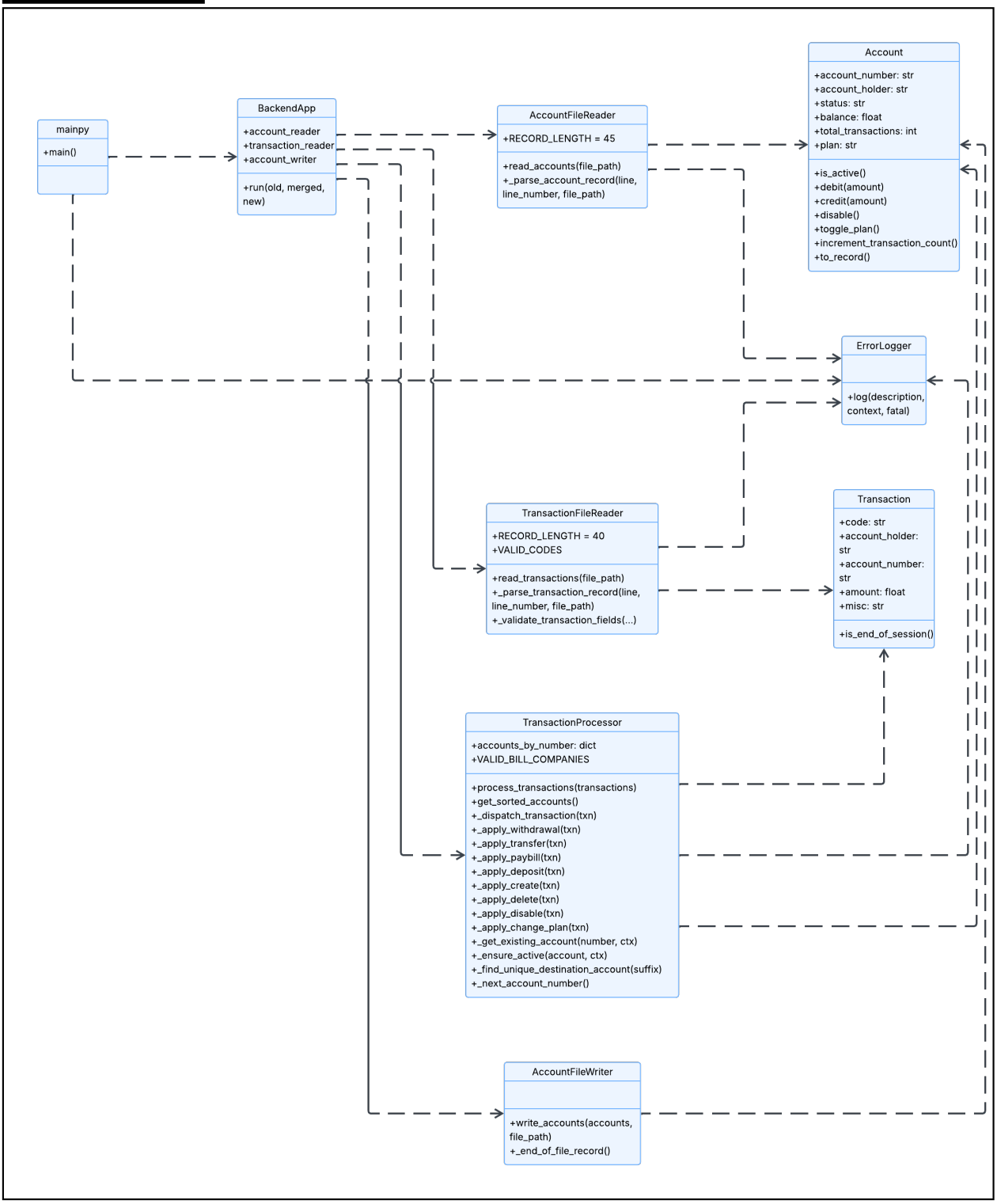
Step	Input	Output
1. Read BAF	Old Master Accounts File	list[Account]
2. Read ATF	Merged Transaction File	list[Transaction]
3. Process	list[Account], list[Transaction]	list[Account] (updated, sorted)
4. Write BAF	list[Account]	New Master Accounts File

Transaction codes handled by the processor:

Code	Operation	Effect
01	Withdrawal	Debit account; reject if disabled or insufficient funds
02	Transfer	Debit source, credit destination (found by 2-digit suffix)
03	Pay Bill	Debit account; validate company code (EC, CQ, or FI)
04	Deposit	Credit account; reject if disabled
05	Create	Add new active account with next available number
06	Delete	Remove account from the in-memory collection
07	Disable	Set account status to D (disabled)
08	Change Plan	Toggle plan between SP and NP
00	End of Session	Structural marker — skipped during processing

4. UML Class Diagram

UML Class Diagram - Phase 4 Back End Banking System



5. Class and Method Descriptions

File / Class	Method	Intention
main.py	main()	Entry point: validates command-line arguments, instantiates BackendApp, and triggers the pipeline.
BackendApp (backend_app.py)	BackendApp()	Constructor: creates one AccountFileReader, one TransactionFileReader, and one AccountFileWriter.
	run(old, merged, new)	Executes the four-step pipeline: read accounts, read transactions, process, write.
Account (account.py)	Account(...)	Constructor: stores account number, holder name, status, balance, transaction count, and plan.
	is_active()	Returns True when the account status is A (active).
	debit(amount)	Subtracts the given amount from the balance.
	credit(amount)	Adds the given amount to the balance.
	disable()	Sets account status to D (disabled).
	toggle_plan()	Switches plan from SP to NP or NP to SP.
	increment_transaction_count()	Increases the cumulative transaction counter by one.
	to_record()	Returns the account formatted as a 45-character fixed-width BAF line.
Transaction (transaction.py)	Transaction(...)	Constructor: stores transaction code, account holder, account number, amount, and misc field.
	is_end_of_session()	Returns True when the transaction code is 00.
AccountFileReader (read.py)	read_accounts(file_path)	Opens the BAF, validates each 45-character line, and returns a list of Account objects.
	_parse_account_record(line, n, path)	Extracts and validates all fields from one BAF line; returns an Account.

File / Class	Method	Intention
TransactionFileReader (read_transactions.py)	read_transactions(file_path)	Opens the ATF, validates each 40-character line, and returns a list of Transaction objects (end-of-session records excluded).
	_parse_transaction_record(line, n, path)	Extracts and validates all fields from one ATF line; returns a Transaction.
	_validate_transaction_fields(...)	Applies code-specific field rules (e.g., company code for paybill, suffix format for transfer).
TransactionProcessor (process.py)	TransactionProcessor(accounts)	Constructor: indexes the account list by account number for O(1) lookup.
	process_transactions(transactions)	Dispatches each transaction to its handler; returns the updated account list sorted by number.
	get_sorted_accounts()	Returns all accounts sorted ascending by account number.
	_dispatch_transaction(txn)	Routes one transaction to the correct handler based on its code.
	_apply_withdrawal(txn)	Debits account; skips if account missing, disabled, or balance insufficient.
	_apply_transfer(txn)	Debits source and credits destination found by 2-digit suffix match.
	_apply_paybill(txn)	Debits account for a bill payment; validates company code.
	_apply_deposit(txn)	Credits account; skips if account missing or disabled.
	_apply_create(txn)	Adds a new active account with the lowest unused account number.
	_apply_delete(txn)	Removes account from the in-memory collection.
	_apply_disable(txn)	Sets account status to disabled.
	_apply_change_plan(txn)	Toggles account plan between SP and NP.
	_get_existing_account(number, ctx)	Returns account by number or logs a non-fatal error if not found.

File / Class	Method	Intention
	_ensure_active(account, ctx)	Returns True if active; logs a non-fatal error and returns False if disabled.
	_find_unique_destination_account(suffix)	Returns the single account whose number ends with the 2-digit suffix; logs error if zero or multiple matches.
	_next_account_number()	Returns the lowest unused account number string (0000199998), or None if all are taken.
AccountFileWriter (write.py)	write_accounts(accounts, file_path)	Writes all account records as 45-character lines followed by the END_OF_FILE sentinel.
	_end_of_file_record()	Returns the fixed-width sentinel string for the last line of the BAF.
ErrorLogger (print_error.py)	log(description, context, fatal)	Prints a formatted error message; calls sys.exit(1) when fatal is True.

6. How to Run

Run from the terminal inside the src/ folder with three positional arguments:

```
    python3 main.py <old_master_accounts_file> <merged_transaction_file>
<new_master_accounts_file>
```

Example using the provided test files:

```
    python3 main.py ../tests/old_master_accounts.txt
../tests/merged_transactions.txt ../tests/new_master_accounts.txt
```

The three arguments are:

Argument	Description
old_master_accounts_file	Path to the Old Master Bank Accounts File (BAF) — 45-character fixed-width format.
merged_transaction_file	Path to the merged Accumulated Transaction File (ATF) — 40-character fixed-width format.
new_master_accounts_file	Output path for the New Master Bank Accounts File. Created or overwritten by the program.

Fatal errors (malformed input file) print an error message and exit with status 1. Non-fatal errors (individual invalid transactions) print a warning message and continue processing the remaining transactions.